An Analysis of XGBoost

XGBoost

XGBoost -- Part 1

Clever penalization of trees A proportional shrinking of leaf nodes Newton Boosting Extra randomization parameter Implementation on single, distributed systems and out-of-core computation Automatic feature selection Theoretically justified weighted quantile sketching for efficient computation Parallel tree structure boosting with sparsity Efficient cacheable block structure for decision tree training

XGBoost -- Part 2

The algorithm XGBoost works as Newton–Raphson in function space unlike gradient boosting that works as gradient descent in function space, a second order Taylor approximation is used in the loss function to make the connection to Newton–Raphson method. A generic unregularized XGBoost algorithm is:

XGBoost -- Part 3

Learning rate (also known as the "step size" or “"shrinkage"), it is a number between 0 and 1 (default is 0.3), which determines the rate the algorithm learns from each iteration. n_estimators, sets the number of trees to be built in the ensemble, where more trees generally increases the complexity of the model, but can lead to overfitting with too many trees. Gamma (also known as Lagrange multiplier or the minimum loss reduction parameter), controls the minimum amount of loss reduction required to make a further split on a leaf node of the tree. The default in XGBoost is 0 . max_depth, represents how deeply each tree in the boosting process can grow during training, where the default is 6.

XGBoost -- Part 4

History XGBoost initially started as a research project by Tianqi Chen as part of the Distributed (Deep) Machine Learning Community (DMLC) group at the University of Washington. Initially, it began as a terminal application which could be configured using a libsvm configuration file. It became well known in the ML competition circles after its use in the winning solution of the Higgs Machine Learning Challenge. Soon after, the Python and R packages were built, and XGBoost now has package implementations for Java, Scala, Julia, Perl, and other languages. This brought the library to more developers and contributed to its popularity among the Kaggle community, where it has been used for a large number of competitions. It was soon integrated with a number of other packages making it easier to use in their respective communities. It has now been integrated with scikit-learn for Python users and with the caret package for R users. It can also be integrated into Data Flow frameworks like Apache Spark, Apache Hadoop, and Apache Flink using the abstracted Rabit and XGBoost4J. XGBoost is also available on OpenCL for FPGAs. An efficient, scalable implementation of XGBoost has been published by Tianqi Chen and Carlos Guestrin. While the XGBoost model often achieves higher accuracy than a single decision tree, it sacrifices the intrinsic interpretability of decision trees. For example, following the path that a decision tree takes to make its decision is trivial and self-explained, but following the paths of hundreds or thousands of trees is much harder.

XGBoost -- Part 5

XGBoost (eXtreme Gradient Boosting) is an open-source software library which provides a regularizing gradient boosting framework for C++, Java, Python, R, Julia, Perl, and Scala. It works on Linux, Microsoft Windows, and macOS. From the project description, it aims to provide a "Scalable, Portable and Distributed Gradient Boosting (GBM, GBRT, GBDT) Library". It runs on a single machine, as well as the distributed processing frameworks Apache Hadoop, Apache Spark, Apache Flink, and Dask. XGBoost gained much popularity and attention in the mid-2010s as the algorithm of choice for many winning teams of machine learning competitions.